book-qr-manager 学習ノート --- 2026年9月19日

> 実際の学習は日をまたいで2026年9月22日まで行ったが、2026年9月19日開始分として記録する。

1. 本日の学習内容

今回は、Book APIのCRUDを完成させたあと、**読書状況を管理する `BookStatus` 機能**の実装を開始した。

- Book APIのCRUD完成
- 読書ステータス用 `enum` の作成
- `BookStatus` Entityの作成
- `Book` と `BookStatus` のOneToOneリレーション
- `@JoinColumn` と外部キーの関係
- `BookStatusModule` の作成
- 1つのServiceで複数Repositoryを利用する方法
- `CreateBookStatusDto` の作成
- BookStatus登録API（POST）の作成
- PostgreSQLの自動採番と `RESTART IDENTITY`
- Postman・pgAdminを使った動作確認

2. Book APIのCRUD完成

Bookについて以下のAPIを実装し、すべて動作確認できた。

処理 HTTPメソッド URL

登録 POST `/books` 一覧取得 GET `/books` ID指定取得 GET `/books/:id` 更新 PATCH `/books/:id` 削除 DELETE `/books/:id`

削除処理は削除後のデータを返さない設計なので `Promise<void>` とした。

```
async remove(id: number): Promise<void> {
  const book = await this.findOne(id);
  await this.bookRepository.remove(book);
}
```

`void` は「返す値がない」、`Promise<void>` は「非同期処理が完了するが値は返さない」という意味。

3. BookStatusテーブルの設計

読書状況は次の情報で管理する。

```
book_status
------------------------
id
```

```
book_id
purchase_date
start_date
finish_date
status
created_at
updated_at
```

`book_id` は対象書籍、`purchase_date` は購入日、`start_date` は読書開始日、`finish_date` は読了日、`status` は現在の読書状況を表す。

4. ReadingStatus enum

自由な文字列ではなく、決められた読書状態だけを扱うためenumを作成した。

```
export enum ReadingStatus {
  WANT_TO_READ = 'want_to_read',
  UNREAD = 'unread',
  READING = 'reading',
  COMPLETED = 'completed',
}
```

`status: string` よりも `status: ReadingStatus` とすることで、「読書ステータスを表す値」という意図が型として明確になる。EntityとDTOの両方で利用した。

5. BookStatus Entity

中心となる実装は以下。

```
@Entity()
export class BookStatus {
  @PrimaryGeneratedColumn()
  id: number;

  @OneToOne(() => Book, (book) => book.book_status)
  @JoinColumn({ name: 'book_id' })
  book: Book;

  @Column({ type: 'date', nullable: true })
  purchase_date: Date | null;

  @Column({ type: 'date', nullable: true })
  start_date: Date | null;

  @Column({ type: 'date', nullable: true })
  finish_date: Date | null;

  @Column({ type: 'enum', enum: ReadingStatus })
  status: ReadingStatus;
```

```
  @CreateDateColumn()
  created_at: Date;

  @UpdateDateColumn()
  updated_at: Date;
}
```

6. BookとBookStatusのOneToOne

1冊のBookに1つのBookStatusを対応させるため、1対1の関係にした。

BookStatus側：

```
@OneToOne(() => Book, (book) => book.book_status)
@JoinColumn({ name: 'book_id' })
book: Book;
```

Book側：

```
@OneToOne(() => BookStatus, (bookStatus) => bookStatus.book)
book_status: BookStatus;
```

これで `Book.book_status` と `BookStatus.book` の両方向から関係をたどれる。

`@JoinColumn` の役割

外部キー `book_id` を持つのはBookStatus側なので、BookStatus側に `@JoinColumn` を書く。

```
@JoinColumn({ name: 'book_id' })
book: Book;
```

Entityに `book_id: number` と直接定義しなくても、TypeORMによってPostgreSQL側には `book_id` が作成される。

外部キーを持つBookStatus側が今回の**所有側**になる。

7. PostgreSQLでテーブルを確認

NestJS起動後、pgAdminで `book_status` が作成されていることを確認した。

```
book_status
├── id
├── purchase_date
├── start_date
```

```
├── finish_date
├── status
├── created_at
├── updated_at
└── book_id
```

`status` はPostgreSQL側でもenum型として作られた。またOneToOneのため、同じBookに複数のBookStatusが紐付かないよう `book_id` の一意性を保証する制約も作成された。

8. BookStatusModule

```
@Module({
  imports: [TypeOrmModule.forFeature([BookStatus, Book])],
  providers: [BookStatusService],
  controllers: [BookStatusController],
})
export class BookStatusModule {}
```

さらに `AppModule` の `imports` に `BookStatusModule` を追加した。

9. 1つのServiceで2つのRepositoryを使う

BookStatus登録ではBookStatusを保存するだけでなく、受け取った `book_id` からBookを検索する必要がある。

```
constructor(
  @InjectRepository(BookStatus)
  private readonly bookStatusRepository: Repository<BookStatus>,

  @InjectRepository(Book)
  private readonly bookRepository: Repository<Book>,
) {}
```

役割は以下。

```
BookStatusRepository → book_statusを操作
BookRepository       → bookを検索
```

そのためModule側でも、

```
TypeOrmModule.forFeature([BookStatus, Book])
```

として両Entityを登録する。

10. CreateBookStatusDto

```
export class CreateBookStatusDto {
  book_id: number;
  purchase_date?: Date;
  start_date?: Date;
  finish_date?: Date;
  status: ReadingStatus;
}
```

日付は状況によってまだ存在しない場合があるため任意項目とした。

11. DTOのbook_idとEntityのbook

今回の重要ポイント。

DTOは、

```
book_id: number;
```

を受け取るが、Entityは、

```
book: Book;
```

を持っている。

そのため `book_id` をそのままEntityへ渡すのではなく、まずBookを検索する。

```
const book = await this.bookRepository.findOneBy({
  id: createBookStatusDto.book_id,
});
```

存在しなければ404を返す。

```
if (!book) {
  throw new NotFoundException(
    `Book with ID ${createBookStatusDto.book_id} not found`,
  );
}
```

実際に存在しないBook IDを指定した際、404が返り、存在チェックが正常に動作することを確認した。

12. BookStatusの作成と保存

```
  const bookStatus = this.bookStatusRepository.create({
    book: book,
    purchase_date: createBookStatusDto.purchase_date,
    start_date: createBookStatusDto.start_date,
    finish_date: createBookStatusDto.finish_date,
    status: createBookStatusDto.status,
  });

  return this.bookStatusRepository.save(bookStatus);
```

`book: book` の左側はBookStatus Entityのプロパティ、右側はBookRepositoryから取得したBook Entity。

処理の流れ：

```
DTOの book_id
    ↓
BookRepositoryで検索
    ↓
Book Entityを取得
    ↓
BookStatus.book に設定
    ↓
TypeORM
    ↓
DBでは book_id として保存
```

`repository.create()` はEntityオブジェクトを作る処理、`repository.save()` はDBへ保存する処理であり、役割が異なる。

13. BookStatus登録API

```
@Controller('book-statuses')
export class BookStatusController {
  constructor(private readonly bookStatusService: BookStatusService) {}

  @Post()
  async create(
    @Body() createBookStatusDto: CreateBookStatusDto,
  ): Promise<BookStatus> {
    return this.bookStatusService.create(createBookStatusDto);
  }
}
```

これで `POST /book-statuses` から登録できる。

全体の流れ：

```
POST /book-statuses
        ↓
BookStatusController
        ↓
CreateBookStatusDto
        ↓
BookStatusService
        ↓
book_idからBookを検索
        ↓
Bookが存在するか確認
        ↓
BookStatus Entityを作成
        ↓
BookStatusRepository.save()
        ↓
PostgreSQL
```

14. 登録テスト

最終的に以下のようなリクエストを送信した。

```
{
  "book_id": 1,
  "status": "reading",
  "purchase_date": "2026-09-01",
  "start_date": "2026-09-22"
}
```

pgAdminでは、

```
id              1
book_id         1
purchase_date   2026-09-01
start_date      2026-09-22
finish_date     null
status          reading
```

として正常に登録された。

15. PostgreSQLのIDが1から始まらなかった理由

`@PrimaryGeneratedColumn()` のIDはPostgreSQLが自動採番する。

```
INSERT → id=1
DELETE → id=1を削除
```

```
INSERT → id=2
```

`DELETE` はデータを削除しても採番情報をリセットしない。そのためIDに欠番があっても正常。

今回は開発用テストデータだったため、

```
TRUNCATE TABLE book RESTART IDENTITY CASCADE;
```

でリセットした。

- `TRUNCATE TABLE book` → データを空にする
- `RESTART IDENTITY` → 自動採番をリセット
- `CASCADE` → bookを参照する関連データにも作用する

その後、BookとBookStatusをID=1から正常に登録できた。

> 本番環境で「IDをきれいに並べる」目的で安易に実行するものではない。IDの欠番自体は問題ではない。

16. 今日特に覚えておきたいポイント

1. **リレーションプロパティにはEntityを持たせる。** `book: Book` のように定義する。
2. **`@JoinColumn` がDB上のFKカラムにつながる。** `book: Book` と書いていてもDBでは `book_id` が作られる。
3. **外部キーを持つ側が所有側。** 今回はBookStatus側。
4. **DTOとEntityは必ずしも同じ形ではない。** `book_id` を `book: Book` に変換するような処理をServiceで行うことがある。
5. **Serviceは複数Repositoryを利用できる。**
6. **`create()` と `save()` は別物。** createはEntity作成、saveはDB保存。
7. **DELETEしてもID採番は戻らない。** 欠番は正常。

17. 次回やること

次回はBookStatusの登録APIに続いて、取得・更新・削除機能を実装してCRUD完成を目指す。

ただしBookStatusの取得には、

```
BookStatus自身のIDから取得する
```

方法と、

```
Book IDからその本のBookStatusを取得する
```

方法が考えられる。

そのため、次回はまず**書籍管理アプリとしてどのURL・取得方法が使いやすいかを整理してから実装する。**

18. 本日の到達点

単純な1テーブルのCRUDから一歩進み、

```
Book 1 ──── 1 BookStatus
```

というリレーションをEntity・TypeORM・PostgreSQL・APIまでつなげることができた。

特に、

```
book_idをAPIで受け取る
       ↓
BookRepositoryでBookを取得
       ↓
BookStatus.bookへ設定
       ↓
TypeORMがbook_idとしてDBへ保存
```

という流れは、今後ReviewやGenreなど複数テーブルを扱う際にも重要になる。

次回開始地点：BookStatusの取得API設計 → CRUD実装